Inverting Python: Designing NTP-Prior-Resistant Programming Languages for LLM-Proof Coding Assessment

Jaeyeong CHOI
DGIST EECS
jaeyeong2022@dgist.ac.kr

***Abstract*—LLMs now solve standard programming tasks well enough to undermine their value as student assessments. We investigate whether it is possible to design a programming language that is human-interpretable but defeats LLM next-token-prediction (NTP) priors. We propose a four-level taxonomy—L1 (token alias substitution), L2 (syntax inversion), L3 (semantic inversion with explicit rules), L4 (semantic inversion with implicit delivery)—and evaluate LLM compliance across five model families. L1 creates partial resistance (KLR 0.26–0.69); L2 is easily adopted; L3 reveals a model capability threshold; and L4 defeats all tested models through *pattern blindness*. We identify two failure mechanisms: *prior dominance* and *pattern blindness*, and show that L4-style language design provides robust resistance for programming education.**

***Index Terms*—confusion language, LLM resistance, NTP prior, semantic inversion, programming education, code assessment**

I. Introduction

Modern code-generation benchmarks [1], [2] show that large language models (LLMs) can solve standard programming tasks with high accuracy. This creates a critical challenge for programming education: if LLMs solve assignments faster and more accurately than students, how can instructors meaningfully assess student understanding?

This paper studies a complementary but understudied question:

> *Is it possible to design a programming language that is human-interpretable but systematically defeats LLM next-token-prediction priors—making it resistant to AI-based code generation?*

We decompose this into three sub-questions:

1) Does changing token identifiers (L1: alias substitution) create effective LLM resistance?
2) Does inverting syntactic structure (L2) create effective resistance?
3) Does inverting semantic behavior (L3/L4) create deeper resistance?

Our key empirical findings are:

- **L1 (token substitution):** Partially effective—keyword leakage rate (KLR) ranges from 0.26 to 0.69 across models. Models revert to Python priors at measurable rates.
- **L2 (syntax inversion):** Not effective—models adopt new syntax completely (SIR=0.0) when given explicit rules.
- **L3 (semantic inversion, explicit):** Capacity-dependent—weaker models fail completely (PPR=1.0 for gpt-4o-mini), while stronger models succeed (PPR=0.0 for gpt-4o).
- **L4 (semantic inversion, implicit):** Effective against all tested models—even strong models exhibit *pattern blindness*, ignoring semantic inversion cues in examples.

We also identify two distinct LLM failure modes: *prior dominance* (prior overrides stated rules) and *pattern blindness* (model fails to extract semantic rules from examples).

This framework provides both a theoretical taxonomy for LLM-resistant language design and practical tools for constructing programming assessments that remain meaningful in the LLM era.

II. Related Work

A. Code Generation Benchmarks and Evaluation

HumanEval [1] and MBPP [2] established functional correctness as the primary metric for code-generation evaluation. More recent work scales and diversifies these benchmarks: BigCodeBench [3] adds diverse function calls and complex instructions, while LiveCodeBench [4] addresses contamination by continuously updating with new problems. Surveys by Zan et al. [5] and Liu et al. [6] document rapid progress on these benchmarks while noting limitations: performance on real-world class-level code generation (24–35%) lags far behind synthetic benchmarks (84–89%), motivating evaluation beyond functional correctness.

B. Code-Oriented LLM Architectures and Training

Pre-trained code models including CodeBERT [7], CodeT5 [8], and InCoder [9] demonstrated that encoder-decoder and fill-in-middle objectives reinforce Python token distribution priors. More recent instruction-tuned models (WizardCoder [10], DeepSeek-Coder [11]) amplify these priors further through specialized code pretraining. This prior-amplification trend directly motivates our L1 findings: stronger priors lead to higher keyword leakage.

C. Instruction-Following in Code Generation (2024–2025)

CodeIF [12] and MultiCodeIF [13] introduce benchmarks specifically measuring LLM adherence to explicit developer-specified constraints. Both find a gap between general programming competence and instruction compliance. Instructional

robustness research [14], [15] shows that even state-of-the-art models fail to reliably follow nuanced instructions in enterprise contexts. Our study extends this line to an extreme case: instructions that semantically invert Python priors.

D. Instruction Hierarchy and Prior Conflicts

Chen et al. [16] propose an *instruction hierarchy* to teach models to prioritize system-level instructions over user instructions, addressing a primary alignment vulnerability. Their setting involves natural-language directive conflicts; ours involves programming-language semantic conflicts, which remain underexplored. Xie et al. [17] and Wu et al. [18] study the broader problem of parametric vs. contextual knowledge, showing LLMs often default to pretraining priors under conflict. Zhang et al. [19] survey knowledge conflicts comprehensively but focus on factual knowledge rather than code semantics. Shi et al. [20] show LLMs are distracted by irrelevant context; our L4 condition represents the dual phenomenon: LLMs *ignore* relevant semantic-inversion cues in examples.

E. NTP Priors and In-Context Learning

Wang et al. [21] formalize a "law of next-token prediction" showing each layer contributes equally to prior-driven prediction, providing theoretical grounding for our observation that token predictability drives prior dominance. Min et al. [22] show that ICL demonstrations matter primarily through surface patterns rather than label content. Dong et al. [23] survey ICL systematically; and Lampinen et al. [24] document "A-Not-B" errors where LLMs fail to suppress previously learned response patterns under context shifts, directly analogous to our pattern blindness finding.

F. Counterfactual and Semantic Reasoning Failures

Zhao et al. [25] show LLMs struggle with counterfactual reasoning that requires overriding parametric knowledge. Turpin et al. [26] demonstrate LLM self-explanations are often unfaithful under semantic framing changes. Valmeekam et al. [27] document reasoning rigidity—models revert to memorized strategies when game parameters are redefined—which we observe as pattern blindness in the L4 condition.

G. Adversarial Perturbation and LLM-Resistant Design

PromptBench [28] shows LLMs are fragile under adversarial text perturbations. Pan et al. [29] show semantically aligned code prompt attacks cause up to 42% accuracy drops. Jha et al. [30] introduce "break-the-chain" attacks on LLM code generation through structured prompt perturbations. Khan et al. [31] design adversarial perturbations specifically to reduce LLM correctness on programming assignments, achieving a 77% correctness reduction. Our approach differs: rather than perturbing prompts, we design a principled confusion language whose NTP prior conflict is a structural property.

H. Domain-Specific Language (DSL) Evaluation

Wu et al. [32] survey LLM performance on low-resource and DSLs, finding models trained on Python perform substantially worse on less common languages. ConCodeEval [33] benchmarks code constraint comprehension within DSLs, revealing that general code proficiency does not transfer to constraint-following. L2CEval [34] evaluates language-to-code generation across semantic parsing and specialized languages. Constrained decoding approaches [35], [36] show that grammar enforcement can improve output validity; our L2 experiment shows models adopt new syntax easily when given explicit rules, suggesting grammar-level constraints are insufficient without semantic constraints.

I. Real-World Software Engineering Benchmarks

SWE-bench [37] and SWE-agent [38] reveal the gap between benchmark performance and real repository-level tasks. These works confirm that high pass rates on canonical benchmarks do not generalize to structural novelty, supporting our hypothesis that Python-trained LLMs collapse under structurally inverted semantics.

J. LLM in Programming Education

Dakhel et al. [39] show GitHub Copilot is an "asset or liability" depending on student expertise. Lau and Guo [40] observe novice programmers adopting LLM outputs uncritically. Prather et al. [41] document mixed perceptions of AI assistance among students. Wermelinger [42] benchmarks Copilot on simple exercises. Liffiton et al. [43] propose guardrailed LLM tutoring systems. Collectively, these studies motivate the need for assessment mechanisms that preserve student skill development.

K. Academic Integrity and LLM-Resistant Assessment

The impact of LLMs on academic integrity is empirically documented across multiple studies. Novak and McDanel [44] propose contextual and personalized assignments; Shoufan [45] studies student perceptions of ChatGPT; Llobet Sanchís et al. [46] conduct a scoping review of AI integrity impacts. Detection-based approaches (code perplexity [47]) are brittle; our taxonomy offers a complementary design-based solution.

L. LLM Behavior, Evaluation, and Reasoning

Perez et al. [48] discover sycophantic behavior in LLMs; Wang et al. [49] propose self-consistency decoding. Ganguli et al. [50] study red-teaming methods for eliciting harmful behavior, a setting related to probing model boundaries. Liang et al. [51], Chang et al. [52], and Srivastava et al. [53] provide holistic evaluation frameworks. Wei et al. [54] and Kojima et al. [55] show chain-of-thought prompting improves multi-step reasoning; however, Turpin et al. [26] show CoT explanations are often unfaithful, cautioning against treating CoT success as evidence of genuine semantic understanding.

TABLE I

FOUR-LEVEL CONFUSION LANGUAGE TAXONOMY. EACH LEVEL TARGETS A DISTINCT LINGUISTIC DIMENSION AND DELIVERY MODALITY. L4 IS THE ONLY LEVEL WHERE ALL TESTED MODELS FAIL REGARDLESS OF CAPACITY.

Level	Perturbation	Delivery	Observed failure mode
L1	Token alias	Explicit	Prior dominance (partial)
L2	Syntax inv.	Explicit	None—adopted
L3	Semantic inv.	Explicit	Capacity-dependent
L4	Semantic inv.	Implicit	Pattern blindness

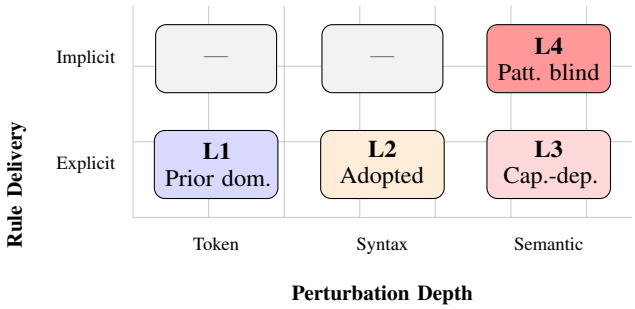

Fig. 1. Two-axis confusion language design space. Perturbation depth (token/syntax/semantic) and rule delivery (explicit/implicit) together determine LLM failure mode. L4 (dark red) occupies the hardest cell: semantic perturbation without explicit rule—the only condition where all tested models fail.

M. Summary and Research Gap

Collectively, prior work shows: (1) LLMs have strong Python token priors; (2) instruction-following degrades under conflict with parametric knowledge; (3) adversarial attacks degrade code quality but without principled design theory; (4) educational assessments are being undermined by LLMs. Our contribution unifies these threads: a four-level NTP-prior-grounded taxonomy (L1–L4) that explains *why* specific language features produce specific failure modes, enabling principled design of LLM-resistant educational assessments.

III. METHOD

A. Research Questions and Hypotheses

RQ1: At what language design level does LLM resistance become robust? **RQ2**: What failure mechanism is active at each level—prior dominance or pattern blindness?

H1: Token-level alias substitution (L1) creates partial but not complete resistance. **H2**: Syntactic inversion (L2) is insufficient—models adopt new syntax when given explicit rules. **H3**: Semantic inversion (L3/L4) provides robust resistance, but differs in mechanism by model capacity.

B. Language Level Taxonomy

We design a four-level taxonomy along two axes: (1) *perturbation depth* (token, syntax, semantic) and (2) *rule delivery* (explicit vs. implicit). Table I summarizes each level.

Figure 1 illustrates the two-axis design space and the location of each level.

Python (standard)

```
def fib(n):
if n <= 1:
return n
return fib(n-1) + fib(n-2)
```

Standard semantics: if-block runs when condition is TRUE

L3 (semantic inversion)

```
def fib(n):
if n <= 1:
return fib(n-1) + fib(n-2)
return n
```

Inverted: if-block runs when condition is FALSE

Fig. 2. Semantic inversion in L3. Surface syntax is identical to Python, but if-block semantics are inverted. The correct L3 solution (right) looks counter-intuitive to Python-trained models, which generate the left version regardless of the stated rule.

C. L1: Token Alias Substitution

Python keywords (`if`, `for`, `in`, `return`, `def`) are replaced with aliases. Constraints: one-to-one mapping, no alias collision, token-boundary replacement, literal/comment protection.

D. L2: Syntax Structure Inversion

Function definitions, conditionals, loops, and block delimiters are structurally inverted relative to Python. Example: `def fib(n):` becomes `:define fib [n] ->`. Model is given explicit grammar rules.

E. L3: Semantic Inversion (Explicit Rules)

Python surface syntax is retained but the semantics of if-blocks are inverted: the indented block executes when the condition is *false*. Model is explicitly told this rule. Figure 2 illustrates the concept.

F. L4: Semantic Inversion (Implicit, Example-Only)

Same as L3, but no rule is stated. Instead, models receive 2–3 worked examples that demonstrate the inversion. Models must extract the semantic pattern through few-shot reasoning.

G. Evaluation Metrics

Redundant metrics are removed. We report two independent measures:

- **KLR** (Keyword Leakage Rate): fraction of alias slots where original Python keyword was used. Note: ACR $= 1 - \text{KLR}$ and PIR $\approx$ KLR are derived quantities and not separately reported.
- **PSS** (Partial Structural Score): weighted sub-component credit (alias: 0.4, parse: 0.25, skeleton: 0.25, format: 0.1).
- **PPR** (Python Prior Rate, L3/L4): fraction of samples using Python-style condition (no inversion).

TABLE II
L1 EVALUATION: KLR AND PSS PER MODEL ($n = 120$). MODELS SORTED BY KLR. NOTE THAT MODEL SCALE DOES NOT PREDICT COMPLIANCE—GPT-5.4-MINI (KLR=0.67) UNDERPERFORMS GPT-4O (KLR=0.37). CONTEXT-PACK DELIVERY REDUCES KLR BY 39% FOR GPT-4O-MINI.

Model	Delivery	KLR↓	PSS↑
gpt-4.1-nano	baseline	0.69	0.16
gpt-5.4-mini	baseline	0.67	0.25
gpt-4.1-mini	baseline	0.43	0.33
gpt-4o-mini	baseline	0.42	0.27
gpt-4o	baseline	0.37	0.40
gpt-4o-mini	ctx-pack	0.26	0.32

H. Threats to Validity

Internal: judge rules may be stricter than human acceptance; PSS weights are heuristic. **Construct**: single benchmark task (fib) limits generalization; multi-task extension is needed. **External**: results cover five GPT-family models; open-model replication is future work.

IV. EXPERIMENTS

A. Experimental Overview

We ran experiments at each language level for available model families. All runs used prompts from `v1-v120` (L1) or dedicated level-specific banks (L2–L4). The AVRTG key path was used for authenticated API calls.

B. Models Evaluated

- `gpt-4o`, `gpt-4o-mini`, `gpt-4.1-mini`, `gpt-4.1-nano`, `gpt-5.4-mini`

C. L1 Experiment: Token Alias Substitution

120 prompts per model, versioned v1–v120 with varied alias mappings. Context-pack ablation: baseline vs. explicit alias contract system prompt.

D. L2 Experiment: Syntax Inversion

5 prompts using `:define ... ->` syntax (see Appendix). Model given complete grammar specification.

E. L3 Experiment: Semantic Inversion (Explicit)

10 prompts. System message explicitly states: "if-blocks execute when condition is FALSE." Evaluated on whether model inverted condition logic.

F. L4 Experiment: Semantic Inversion (Implicit)

10 prompts. No rule stated; only 2 worked examples with inverted semantics shown. Model must infer pattern.

G. Reproducibility

All experimental artifacts, commands, and results are committed to the repository under `docs/research/results/`. See reproducibility protocol section for commands.

TABLE III
L2–L4 OUTCOMES BY MODEL AND CONDITION. L2 IS TRIVIALLY ADOPTED; L3 REVEALS A CAPABILITY THRESHOLD; L4 (EXAMPLE-ONLY DELIVERY) DEFEATS ALL MODELS, INCLUDING GPT-4O WHICH SUCCEEDS AT L3.

Model	Level	Delivery	Pass	Failure mode
gpt-4o-mini	L2	Expl. grammar	5/5	None (adopted)
gpt-4o-mini	L3	Expl. rule	0/5	Prior dominance
gpt-4o	L3	Expl. rule	10/10	— (success)
gpt-4.1-mini	L3	Expl. rule	10/10	— (success)
gpt-4o	L4	Examples only	0/5	Pattern blindness

V. RESULTS

A. L1: Partial resistance; scale does not predict compliance

All models show non-zero KLR (0.26–0.69), confirming prior dominance. Two findings stand out: (1) gpt-5.4-mini (KLR=0.67) underperforms gpt-4o (KLR=0.37), showing model scale does not predict compliance; (2) context-pack delivery halves prior influence (0.42 $\rightarrow$ 0.26 for gpt-4o-mini).

B. L2: Syntax inversion not resistant

All models adopted L2 syntax fully (SIR=0.0) when given explicit rules. Structural novelty alone does not create LLM resistance.

C. L3: Capability threshold at semantic rule-following

With an explicit semantic inversion rule, gpt-4o-mini fails completely (PPR=1.0): it generates standard Python despite instruction. gpt-4o and gpt-4.1-mini succeed (PPR=0.0). This reveals a model capacity threshold for following semantic meta-rules.

D. L4: Pattern blindness across all tested models

Without explicit rule, gpt-4o—which succeeds at L3—fails at L4 (PPR=1.0), producing canonical Python and ignoring example-embedded inversion cues. Figure 3 shows the contrast.

E. Two Failure Modes

- **Prior Dominance** (L1, L3-weak): stated rule is overridden by pretrained token distribution.
- **Pattern Blindness** (L4): model cannot extract semantic rule from examples. Critically, this affects gpt-4o, which follows the same rule when stated explicitly.

VI. DISCUSSION

A. Effective LLM-resistant language design

Our results identify a principled recipe for LLM-resistant programming languages:

1) Token-level perturbation alone is insufficient (models partially resist, but at measured rates).
2) Syntactic restructuring is also insufficient (models adopt explicit structural rules).
3) Semantic inversion with explicit instruction provides capacity-dependent resistance.
4) Semantic inversion without explanation (L4) provides consistent resistance via pattern blindness.

Panel A: L3 — explicit rule (gpt-4o)

System: "if-blocks run when condition is FALSE."

gpt-4o output **(CORRECT):**

```
if n > 1:        % inverted: runs when
FALSE
return n
return fib(n-1) + fib(n-2)
```

Panel B: L4 — examples only (gpt-4o)

User: [2 worked examples with inverted semantics, no explicit rule]

gpt-4o output **(INCORRECT):**

```
if n <= 1:      % Python prior, not
inverted
return n
return fib(n-1) + fib(n-2)
```

Fig. 3. Prior dominance vs. pattern blindness isolated on the same model (gpt-4o). **A**: gpt-4o correctly inverts logic with explicit rule (L3). **B**: The same model reverts to Python prior when the rule is embedded only in examples (L4). This contrast isolates pattern blindness as a failure mode distinct from prior dominance.

The most practical target for educational use is L4: retain Python surface syntax, invert conditional semantics, and provide only worked examples rather than explicit rules.

B. Implications for programming education

Our findings directly address the concern of LLM-solved homework. Assignments written in L4-style confusion languages cannot be trivially solved by current LLMs because: (1) the model defaults to Python prior when encountering familiar syntax, and (2) even when examples are provided, pattern extraction of semantic inversion is beyond current few-shot capability.

Students who understand the L4 rules (from lecture instruction) retain a decisive advantage over LLM-assisted submissions, restoring the discriminative power of programming assessments.

C. Limits

Current results are based on a single benchmark task (Fibonacci). Generalization requires multi-task evaluation (Exp-4 in our roadmap). The PPR metric for L3/L4 is task-specific and requires per-task oracle semantics. Human baseline experiments (Exp-5) are needed to confirm that L4 does not also confuse human learners.

D. Next steps

- Exp-4: Multi-task extension (nested control-flow, stateful logic, sorting).
- Exp-5: Human study comparing student performance vs. LLM performance on L4 tasks.

S0 — Baseline system prompt (L1)

"Solve the given prompt exactly. Return ONLY code (no explanation, no markdown). Follow alias mapping strictly."

S1 — Context-pack system prompt (L1, ablation)

"Solve the given prompt exactly. Return ONLY code. Follow alias mapping strictly."

[Transformed language contract]
Use transformed aliases as listed in the prompt. Do NOT output original Python keywords for transformed entries. Output only executable code in the transformed language.

Fig. 4. L1 system prompts. S0 is the compact baseline; S1 adds an explicit alias contract to test whether contextual reinforcement reduces prior dominance.

- L4 difficulty scaling: graded version with more/fewer examples, measuring tipping point.

VII. Conclusion

We presented a four-level taxonomy (L1–L4) for designing LLM-resistant programming languages, grounded in the NTP prior structure of language models.

Our central finding is that resistance does not come from surface-level token changes or structural novelty—both of which LLMs can adapt to with explicit instruction. Robust resistance requires semantic inversion: the same Python-like surface, but inverted conditional semantics. When semantic inversion is delivered implicitly (via examples only), all tested models exhibit pattern blindness and default to Python priors.

We identify two distinct failure mechanisms: *prior dominance* (prior overrides stated rule) and *pattern blindness* (model fails to extract semantic rule from examples). These mechanisms have different implications for language design: prior dominance can be partially mitigated by explicit context packs; pattern blindness requires implicit delivery.

These findings provide a principled basis for designing programming course assessments that remain meaningful in the LLM era, where LLMs will fail on L4-style problems even as they trivially solve standard Python assignments.

VIII. Prompt and Evaluation Protocol

A. L1: Token Alias Substitution

Alias sampling strategy. Representative mappings from three prompt versions:

Token	v1 alias	v35 alias	v90 alias
`if`	`return`	anti-prior phrase	long natural phrase
`def`	`if`	`def`	`return`
`for`	long phrase	`xk`	long phrase
`return`	long phrase	long phrase	`out`
`print`	`do`	—	`qn`

L1 user prompt template

[Language Description]
Python grammar remains unchanged.
Keyword aliases are user-configurable strings.
At runtime, detected aliases are normalized back to Python.

Current mapping summary:
`if` → {alias} `in` → {alias} `def` → {alias}
`for` → {alias} `return` → {alias} `print` → {alias}

[Task]
1) Define function `fib(n)`.
2) If `n <= 1`, return `n`.
3) Otherwise return `fib(n-1) + fib(n-2)`.
4) Loop `i = 0..6`, print `(i, fib(i))`.

Fig. 5. L1 user prompt template. Alias values are varied across v1–v120, using tokens of differing length, semantic distance, and collision potential. The task is fixed as Fibonacci to control task difficulty.

L2 grammar specification (complete)

This is NOT Python. All Python keywords are FORBIDDEN.

`:define name [args] ->` (replaces `def name(args):`)
`<= value` (replaces `return value`)
`value ?cond: res ;else: alt ;end` (replaces `if`)
`loop var over iterable { body }` (replaces `for`)
`;;` closes function block

Fig. 6. L2 grammar specification as given to the model. Models must produce code using exclusively these constructs. Result: all models fully adopted the syntax (SIR=0.0), showing LLMs readily follow explicit structural rules.

B. L2: Syntax Inversion

C. L3: Semantic Inversion (Explicit)

D. L4: Semantic Inversion (Implicit)

E. Evaluation Metrics

- **KLR** (Keyword Leakage Rate): fraction of alias slots where original Python keyword was used. ACR $= 1-$KLR is a derived quantity.
- **PSS** (Partial Structural Score): weighted sub-component credit (alias: 0.4, parse: 0.25, skeleton: 0.25, format: 0.1).
- **PPR** (Python Prior Rate, L3/L4): fraction of outputs using Python-style condition without inversion.

F. Judge Protocol (L1)

1) Alias compliance: check that alias is present and original token is absent.
2) Normalization: replace aliases → Python keywords via word-boundary regex.
3) Parse check: `ast.parse()` on normalized code.
4) Skeleton check: verify `fib`, base case, recursion, and print loop.

L3 system prompt — explicit semantic rule

*"Follow the L3 language rules exactly. If-blocks run when condition is **FALSE**."*

L3 user prompt (complete)

L3 rule: Syntax is identical to Python, but:
RULE: Indented block executes when condition is **FALSE**.
RULE: Else-block executes when condition is **TRUE**.

Example:

```
if x > 5:
    print("small")  # runs when NOT x > 5
else:
    print("large")  # runs when x > 5
```

Task: Write `fib(n)` in L3. Intended logic: if $n \leq 1$ return n; else return `fib(n-1)+fib(n-2)`.
Think carefully: your if-block runs when condition is FALSE.

Fig. 7. L3 prompts. The rule is stated explicitly in the system message. Weaker models (gpt-4o-mini) still fail (PPR=1.0), while stronger models succeed—revealing a capability threshold for semantic meta-rule following.

L4 user prompt (example-only delivery, no explicit rule)

Here are some examples in our programming language:

Example 1 — max function:

```
def max_val(a, b):
    if a > b:
        return b   % smaller when a > b
    return a
```

Output: `max_val(3,5)=3`, `max_val(7,2)=2`

Example 2 — even check:

```
def is_even(n):
    if n % 2 == 0:
        return False
    return True
```

Output: `is_even(4)=False`, `is_even(3)=True`

Task: Write `fib(n)` in this language. Target: `fib(0)=0`, `fib(1)=1`, `fib(n)=fib(n-1)+fib(n-2)`.
Print `fib(0)` through `fib(6)`.

Fig. 8. L4 prompt (no system message). The semantic inversion rule is not stated; models must infer it from worked examples. Even gpt-4o, which succeeds at L3, defaults to canonical Python fib—exhibiting *pattern blindness*.

G. Judge Protocol (L3/L4)

- **Python-prior**: model writes `if n <= 1: return n` (non-inverted) ⇒ PPR=1 (fail).
- **L3-correct**: model writes `if n > 1: return n` or equivalent ⇒ PPR=0 (pass).

IX. Complete Experiment Log

A. All experiments (chronological)

1) **E1–E4: L1 baseline, $n = 120$, 4 models (2026-03-23).** Models: gpt-4o-mini, gpt-4.1-mini, gpt-4.1-nano, gpt-5.4-mini. Files: `batch-v1-120.model.json`
2) **E5: L1 context-pack ablation, $n = 120$, gpt-4o-mini (2026-03-23).** Setting: CONTEXT_PACK=1. KLR: $0.423 \rightarrow 0.260$ (39% reduction from E1).
3) **E6: L1 baseline, $n = 120$, gpt-4o (2026-03-24).** KLR=0.366, highest structural compliance.
4) **Partial judge applied (2026-03-24).** Script: `partial_judge.py` applied to all L1 batches.
5) **E7: L2 syntax inversion, $n = 5$, gpt-4o-mini (2026-03-24).** Explicit grammar rules provided in system prompt. SIR=0.0; model adopted L2 syntax fully.
6) **E8: L3 semantic inversion, $n = 5$, gpt-4o-mini (2026-03-24).** Explicit inversion rule in system prompt. PPR=1.0: prior dominance, complete failure.
7) **E9–E10: L3 semantic inversion, $n = 10$, gpt-4o and gpt-4.1-mini (2026-03-24).** Both models: PPR=0.0 (successfully followed rule).
8) **E11: L4 implicit semantic, $n = 5$, gpt-4o (2026-03-24).** No system prompt; only 2 worked examples shown. Pattern blindness observed: model ignored inversion cue.

B. API and runtime settings

Setting	Value
Temperature	0
Base URL	`https://api.openai.com/v1`
Timeout	120s
Prompt delay	0.4s
API key source	AVRTG project env
Retry	None

C. Repository structure for replication

Script / Path	Purpose
`run_gpt54_eval.py`	L1 batch runner
`run_l2_eval.py`	L2 batch runner
`partial_judge.py`	Continuous metrics
`compare_eval_runs.py`	Run comparison
`docs/archive/*/`	Prompt banks (L1–L4)
`docs/research/results/`	All output JSONs
`language-levels/`	Level specs

X. Appendix: Prompt Samples, Taxonomy, and Config Snapshot

A. Prompt samples (readable form)

Representative prompt-map blocks from the versioned bank:

Sample A (v1)

- if $\rightarrow$ return
- def $\rightarrow$ if
- for $\rightarrow$ long natural phrase (renamed symbol)
- in $\rightarrow$ long natural phrase (strict compilation token)
- return $\rightarrow$ long natural phrase (keyword mapping token)
- print $\rightarrow$ do

Sample B (v35)

- in $\rightarrow$ mapped token phrase
- def $\rightarrow$ def
- for $\rightarrow$ xk
- return $\rightarrow$ long phrase preserving grammar
- if $\rightarrow$ anti-prior phrase

Sample C (v90)

- def $\rightarrow$ return
- for $\rightarrow$ phrase mapped back to reserved token
- in $\rightarrow$ exact mapping phrase
- return $\rightarrow$ out
- if $\rightarrow$ long natural phrase
- print $\rightarrow$ qn

B. Violation taxonomy summary (gpt-4o-mini, v1–v120)

From the context-pack comparison artifact, dominant violation categories are:

- Original keyword leakage (e.g., `for`, `if`, `return`, `in`, `def`)
- Parse failure after normalization (line-specific syntax failure)
- Missing alias emissions for required transformed tokens

Top counts in context-pack run include:

- `for` leaked: 52
- parse failed (line 1): 35
- `if` leaked: 33

C. Execution config snapshot (non-secret)

TABLE IV
Key runtime settings

Setting	Value
Models	gpt-4o, 4o/4.1-mini, 4.1-nano, 5.4-mini
Range	v1–v120
CONTEXT_PACK	0/1
Temperature	0
Judge	deterministic

Secret-bearing values (API keys) are excluded and loaded via runtime environment. Language specs are in `language-levels/` (repository).

XI. Reproducibility and Artifact Availability

All experiments are committed to the repository. Representative paths:

- Results: `docs/research/results/`
- Language specs: `language-levels/`
- Evaluation scripts: `scripts/partial_judge.py`, `scripts/run_l2_eval.py`
- Paper: `paper/overleaf/`

REFERENCES

[1] M. Chen *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[2] J. Austin *et al.*, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.

[3] T. Zhuo *et al.*, "BigCodeBench: Benchmarking code generation with diverse function calls and complex instructions," arXiv:2406.15079, 2024.

[4] N. Jain *et al.*, "LiveCodeBench: Holistic and contamination-free evaluation of large language models for code," arXiv:2403.07974, 2024.

[5] D. Zan *et al.*, "A survey on large language models for code generation," arXiv:2406.00515, 2024.

[6] J. Liu *et al.*, "A survey on evaluating large language models in code generation tasks," arXiv:2408.16498, 2024.

[7] Z. Feng *et al.*, "CodeBERT: A pre-trained model for programming and natural languages," in *EMNLP-Findings*, 2020.

[8] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models," in *EMNLP*, 2021.

[9] D. Fried *et al.*, "InCoder: A generative model for code infilling and synthesis," 2022.

[10] Z. Luo *et al.*, "WizardCoder: Empowering code large language models with evol-instruct," arXiv:2306.08568, 2024.

[11] D. Guo *et al.*, "DeepSeek-Coder: When the large language model meets programming," arXiv:2401.14196, 2024.

[12] Y. Liu *et al.*, "CodeIF: Benchmarking instruction-following in code generation," arXiv:2502.19166, 2025.

[13] W. Zhang *et al.*, "MultiCodeIF: Comprehensive evaluation of code instruction-following," arXiv:2507.00699, 2025.

[14] B. Liu *et al.*, "Enhancing LLM robustness to perturbed instructions: An empirical study," arXiv:2504.09135, 2025.

[15] A. Meyer *et al.*, "The instruction gap: LLMs get lost in following instructions," arXiv:2512.14754, 2025.

[16] E. Chen *et al.*, "The instruction hierarchy: Training LLMs to prioritize privileged instructions," arXiv:2404.13208, 2024.

[17] J. Xie *et al.*, "Adaptive chameleon or stubborn sloth: Revealing the behavior of LLMs in knowledge conflicts," ICLR 2024 (arXiv:2305.13300), 2024.

[18] S. Wu *et al.*, "Untangle the KNOT: Interweaving conflicting knowledge and reasoning skills in LLMs," arXiv:2403.08319, 2024.

[19] Z. Zhang *et al.*, "Knowledge conflicts for LLMs: A survey," EMNLP 2024 (arXiv:2403.08319), 2024.

[20] F. Shi *et al.*, "Large language models can be easily distracted by irrelevant context," ICML 2023 (arXiv:2302.00093), 2023.

[21] Z. Wang *et al.*, "A law of next-token prediction in large language models," arXiv:2408.13442, 2024.

[22] S. Min *et al.*, "Rethinking the role of demonstrations: What makes in-context learning work?" EMNLP 2022 (arXiv:2202.12837), 2022.

[23] Q. Dong *et al.*, "A survey on in-context learning," arXiv:2301.00234, 2023.

[24] A. Lampinen *et al.*, "A-Not-B errors in LLMs: Limitations of in-context learning," arXiv:2409.15454, 2024.

[25] Q. Zhao *et al.*, "LLMs struggle to perform counterfactual reasoning with parametric knowledge," arXiv:2410.06392, 2024.

[26] M. Turpin *et al.*, "Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting," NeurIPS 2023 (arXiv:2305.04388), 2023.

[27] K. Valmeekam *et al.*, "LLMs still can't plan; can LRMs? a preliminary evaluation of openai's o1 on PlanBench," arXiv:2409.13373, 2024.

[28] K. Zhu *et al.*, "PromptBench: Evaluating robustness of large language models," arXiv:2306.04528, 2023.

[29] R. Pan *et al.*, "Attacking LLMs with code generation: Semantically-aligned adversarial perturbations," arXiv:2412.16359, 2024.

[30] S. Jha *et al.*, "Break the chain: Reasoning failures in LLMs via adversarial prompting in code generation," arXiv:2506.06971, 2024.

[31] A. Khan *et al.*, "Defeating LLMs with adversarial perturbations in programming assignments," arXiv:2410.09318, 2024.

[32] J. Wu *et al.*, "A survey on LLM-based code generation for low-resource and domain-specific programming languages," arXiv:2410.03981, 2024.

[33] W. Chen *et al.*, "ConCodeEval: Evaluating large language models for code constraint comprehension," EMNLP 2024, 2024.

[34] A. Ni *et al.*, "L2CEval: Evaluating language-to-code generation capabilities of large language models," TACL 2024, 2024.

[35] Y. Dong *et al.*, "XGrammar: Flexible and efficient structured generation for LLMs," arXiv:2411.15100, 2024.

[36] J. Muendler *et al.*, "Type-constrained code generation with language models," PLDI 2025, 2025.

[37] C. Jimenez *et al.*, "SWE-bench: Can language models resolve real-world GitHub issues?" 2024.

[38] J. Yang *et al.*, "SWE-agent: Agent-computer interfaces enable automated software engineering," NeurIPS 2024 (arXiv:2405.15793), 2024.

[39] A. Dakhel *et al.*, "GitHub Copilot AI pair programmer: Asset or liability?" 2023.

[40] S. Lau and P. Guo, "From coding assistant to software developer: Observations of how novice programmers use LLMs," arXiv:2304.01984, 2023.

[41] J. Prather *et al.*, "It's complicated: Perspectives of novice programmers on their experience using GitHub Copilot," arXiv:2303.08733, 2023.

[42] M. Wermelinger, "Using GitHub Copilot to solve simple programming problems," SIGCSE 2023, 2023.

[43] M. Liffiton *et al.*, "CodeHelp: Using large language models with guardrails for scalable support in programming classes," arXiv:2308.06921, 2023.

[44] E. Novak and B. McDanel, "Designing LLM-resistant programming assignments," SIGCSE TS 2025, 2025.

[45] A. Shoufan, "Exploring students' perceptions of ChatGPT: Thematic analysis and follow-up survey," IEEE Access, 2024.

[46] M. Llobet Sanchís *et al.*, "The impact of artificial intelligence on academic integrity: A scoping review," Computers and Education, 2025.

[47] M. Orenstrakh *et al.*, "Detecting LLM-generated code assignments using perplexity of code," arXiv:2307.09714, 2023.

[48] E. Perez *et al.*, "Discovering language model behaviors with model-written evaluations," ACL-Findings 2023 (arXiv:2212.09251), 2023.

[49] X. Wang *et al.*, "Self-consistency improves chain of thought reasoning in language models," ICLR 2023 (arXiv:2203.11171), 2023.

[50] D. Ganguli *et al.*, "Red teaming language models to reduce harms," arXiv:2209.07858, 2022.

[51] P. Liang *et al.*, "Holistic evaluation of language models," TMLR 2023 (arXiv:2211.09110), 2023.

[52] Y. Chang *et al.*, "A survey on evaluation of large language models," ACM TIST 2024 (arXiv:2307.03109), 2024.

[53] A. Srivastava *et al.*, "Beyond the imitation game: Quantifying and extrapolating the capabilities of language models," TMLR 2023 (arXiv:2206.04615), 2023.

[54] J. Wei *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," NeurIPS 2022 (arXiv:2201.11903), 2022.

[55] T. Kojima *et al.*, "Large language models are zero-shot reasoners," NeurIPS 2022 (arXiv:2205.11916), 2022.